Experiment 7:. Random Sampling and Sampling Distribution

Aim:

To explore random sampling from a population and understand the concept of sampling distribution using Python in Jupyter Notebook.

Steps:

1. Generate a Population:

○ Create a population of data with a specified distribution (e.g., normal distribution).

2. Random Sampling:

○ Perform random sampling from the population to create multiple samples of different sizes.

○ Compute sample statistics (mean, standard deviation, etc.) for each sample.

3. Sampling Distribution:

○ Plot histograms or density plots of sample statistics (e.g., sample means).

○ Compare the sampling distribution of the sample statistic (mean) with the population distribution.

4. Central Limit Theorem (Optional):

○ Demonstrate the Central Limit Theorem by showing that as sample size increases, the sampling distribution of the sample mean approaches a normal distribution regardless of the population distribution.

Code:

```
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Generate a population (e.g., normal distribution)
population_mean = 50
```

```
population_std = 10
population_size = 100000
population = np.random.normal(population_mean, population_std, population_size)

# Step 2: Random sampling
sample_sizes = [30, 50, 100] # different sample sizes to consider
num_samples = 1000 # number of samples for each sample size
```



```
sample_means = {}

for size in sample_sizes:
sample_means[size] = []
for _ in range(num_samples):
sample = np.random.choice(population, size=size, replace=False)
sample_means[size].append(np.mean(sample))

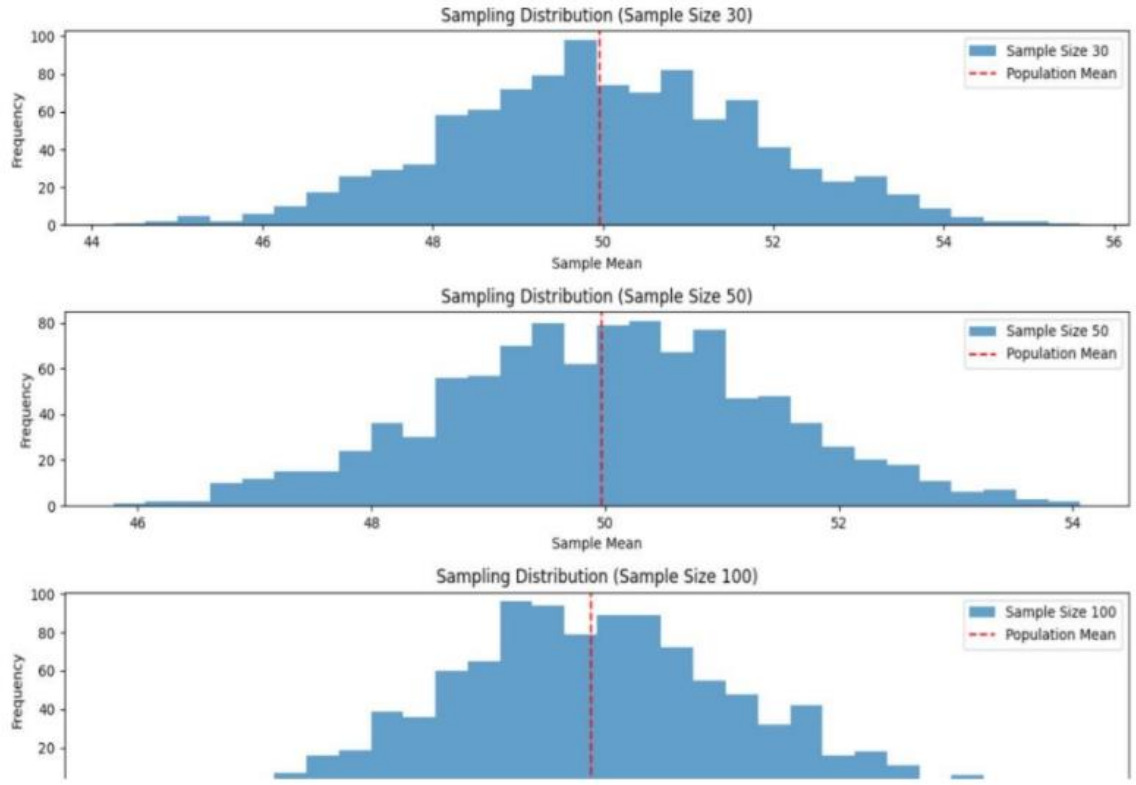
# Step 3: Plotting sampling distributions
plt.figure(figsize=(12, 8))

for i, size in enumerate(sample_sizes):
plt.subplot(len(sample_sizes), 1, i+1)
plt.hist(sample_means[size], bins=30, alpha=0.7, label=f'Sample Size {size}')
plt.axvline(np.mean(population), color='red', linestyle='dashed', linewidth=1.5,
label='Population
Mean')
plt.title(f'Sampling Distribution (Sample Size {size})')
plt.xlabel('Sample Mean')
```

```
plt.ylabel('Frequency')
plt.legend()

plt.tight_layout()
plt.show()
```

Output:



Result:

The above code is executed successfully.